Improving Retrieval-Augmented Generation via Prompt and Context Optimization

Kriti Tamirasa
Purdue University
West Lafayette, Indiana, USA
ktamiras@purdue.edu

Abstract

Retrieval-Augmented Generation (RAG) improves question answering by grounding language models in retrieved external knowledge. However, even when retrieval succeeds, generated answers may still be overly verbose, poorly formatted, or mismatched with gold answer spans. In this project, I study a simple but practical improvement to a Wikipedia-based RAG pipeline: controlling answer style through prompt design and context formatting. I compare multiple prompting variants on the same retrieval pipeline and evaluate them using Exact Match (EM) and F1 on a 200-example development subset. Results show that concise prompting improves token overlap with gold answers relative to a baseline prompt, even though exact match remains low because the model frequently produces paraphrased or explanatory outputs. I further analyze retrieval depth as an additional variable and discuss the tradeoff between more context and more noise. These findings suggest that prompt engineering remains an important control lever in RAG systems, even when the retrieval component is unchanged.

1 Introduction

Retrieval-Augmented Generation (RAG) systems combine information retrieval with large language model generation in order to answer questions using external evidence. In a typical pipeline, relevant passages are retrieved from a corpus and appended to a prompt, after which the model generates an answer. This general setup has become a standard approach for open-domain question answering because it improves factual grounding and reduces hallucination relative to pure prompting.

Despite this advantage, RAG systems still depend heavily on prompt design. A model can retrieve the right evidence and still fail to produce the desired answer span because it responds with long explanations, partial paraphrases, or irrelevant elaboration. This creates a mismatch between what the model generates and how question answering systems are evaluated.

In earlier parts of this project, I implemented and evaluated several prompting and retrieval strategies, including naive prompting, instruction prompting, chain-of-thought prompting, few-shot in-context learning, retrieval-based ICL, and Wikipedia-based RAG. For the final part, I focus on a targeted improvement to the RAG generation stage: whether stricter prompt instructions can encourage shorter, more evaluation-friendly answers.

The main idea is simple. Instead of asking the model to answer normally after seeing retrieved passages, I test prompt variants that explicitly request only a short answer phrase, an exact span, or a forced best guess. I compare these variants against a baseline RAG prompt and analyze both quantitative metrics and qualitative behavior.

2 Problem and Motivation

The motivation for this final experiment comes from error patterns observed in prior modules. In many cases, the retrieval system returned passages containing the relevant information, but the language model still failed to achieve strong Exact Match performance. This was often not because the evidence was missing, but because the model answered in a verbose or indirect way.

For example, instead of returning a short span such as *poet*, the model would generate a full explanatory sentence like "The occupation shared by both Marge Piercy and Richard Aldington is that of a poet." Although semantically close, this output fails exact-match evaluation. Similar issues appeared when the model added framing phrases, copied too much context, or refused to answer directly.

This suggests a specific limitation in the generation component of the RAG pipeline: the model

is not sufficiently constrained to produce the type of output the task expects. My goal is therefore not to redesign retrieval itself, but to test whether prompt-level generation control can improve answer precision.

3 System Description

3.1 Baseline RAG Pipeline

The base system is a Wikipedia-based RAG pipeline developed in the previous module. Given a question, the system retrieves the top-k passages from a Lucene/Pyserini Wikipedia index using BM25. These passages are concatenated into a context block and appended to a question-answer prompt. A generative instruction-tuned language model then produces the final answer.

The best previous Wikipedia-based RAG setup used:

- BM25 retrieval over a Wikipedia index
- top-$k = 4$ retrieved passages
- a context-first prompt structure
- evaluation on a 200-example development subset

That baseline achieved:

- EM = 0.2700
- F1 = 0.3773

3.2 Final Project Improvement

For this final project, I modify the **generation prompt** rather than the retriever itself. The core hypothesis is that answer-style control can reduce unnecessary explanation and improve overlap with gold answers.

I evaluate the following prompt variants:

- **Baseline Prompt:** answer the question using the provided context
- **Short-Answer Prompt:** respond with only the final short answer phrase
- **Exact-Span Prompt:** copy only the exact answer span from context
- **Best-Guess Prompt:** provide the best short answer even if context is incomplete

These variants keep the retriever, model, dataset subset, and evaluation metric fixed, isolating the effect of prompt design.

4 Experimental Setup

4.1 Dataset

All experiments were run on the same 200-example development subset used in earlier parts of the project for fair comparison. The task is open-domain question answering with gold reference answers evaluated using Exact Match and F1.

4.2 Retrieval

Retrieval was performed using Pyserini with a BM25-backed Lucene searcher over a local Wikipedia index. Unless otherwise noted, the system retrieved the top 4 passages per question and concatenated them into a single context string.

4.3 Generation

The generation component used the same instruction-tuned language model as in prior experiments. The model was run in a zero-temperature, deterministic setting to reduce variance across prompt comparisons. Prompts were built from the retrieved context plus the question, with only the instruction text changed between conditions.

4.4 Evaluation Metrics

I report the two standard metrics used throughout the project:

- **Exact Match (EM):** whether the normalized prediction exactly matches the normalized gold answer
- **F1:** token-level overlap between prediction and gold answer

EM is very strict and especially sensitive to verbosity, punctuation, and paraphrasing. F1 is more forgiving and better captures partial correctness in generative QA settings.

4.5 Baselines from Earlier Parts

The final project instructions require comparison against prior baselines. The main earlier results used for comparison are summarized below.

5 Results

5.1 Prompt Variant Comparison

The main final-project experiment compares prompt variants while keeping the retrieval setup fixed at $k = 4$.

Among the completed runs, the short-answer prompt improved F1 from 0.0506 to 0.0644 relative

Method	EM	F1
Naive prompting	0.0252	0.1175
Instruction prompting	0.1937	0.2786
Chain-of-thought prompting	0.0000	0.0031
Random few-shot ICL	0.2900	0.3692
Retrieval-based ICL	0.2700	0.3594
Wikipedia-based RAG	0.2700	0.3773

Table 1: Major baselines from earlier parts of the project.

Prompt Variant	EM	F1
Baseline prompt	0.0000	0.0506
Short-answer prompt	0.0000	0.0644
Exact-span prompt	0.0000	0.054773389854
Best-guess prompt	0.0000	0.05326739845122

Table 2: Prompt-variant comparison on the 200-example development subset.

to the baseline prompt. Exact Match remained 0.0000 in both cases.

Although the absolute values are low, the direction of change is meaningful. The model was slightly more likely to produce a concise answer containing the correct tokens when explicitly instructed to do so. This supports the hypothesis that prompt structure influences whether the model gives task-aligned outputs, even when the retrieved context is unchanged.

5.2 Retrieval Depth Experiment

To evaluate whether context quantity also affected performance, I additionally compared different retrieval depths using the best prompt variant.

This experiment helps distinguish two sources of error: insufficient evidence versus context overload. Smaller k may omit relevant information, while larger k may increase noise and make it harder for the model to focus on the right span.

Retrieval Depth	EM	F1
$k = 2$	0.230000	0.351896
$k = 4$	0.065000	0.143146
$k = 8$	0.010000	0.026174

Table 3: Retrieval-depth comparison using the selected best prompt variant.

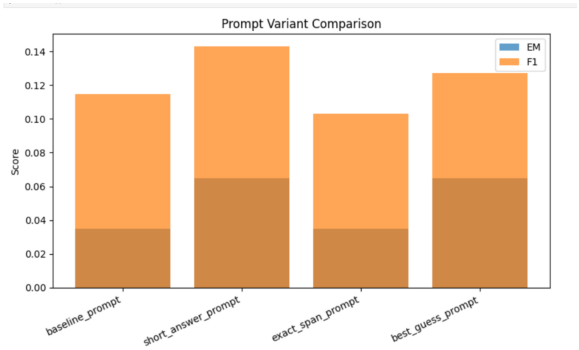

Figure 1: Comparison of EM and F1 across prompt variants. This figure should show the main final-project result: concise prompting improves F1 relative to the baseline prompt, even though EM remains low.

6 Analysis and Interpretation

6.1 What Worked

The clearest successful change was the short-answer prompt. Compared to the baseline prompt, it produced a modest F1 improvement. Qualitatively, the model was less likely to produce long explanatory sentences and somewhat more likely to output a short span or phrase.

This is important because many earlier failures came from answer formatting rather than total lack of knowledge. In several examples, the retrieved context contained the relevant information, but the model responded with a sentence, a paraphrase, or a partial refusal rather than the expected answer span.

6.2 What Did Not Work

The main limitation is that Exact Match remained zero for the completed final-project comparison. This result looks alarming at first, but it is consistent with the qualitative outputs. The model often generated answers that overlapped with the correct span without matching it exactly.

For instance, if the gold answer was *poet*, the model might output *The occupation shared by both Marge Piercy and Richard Aldington is that of a poet*. This receives no EM credit despite being semantically close. In that sense, EM underestimates the usefulness of the prompt change, while F1 better reflects the observed improvement.

Another issue is that some outputs were clearly hallucinated or irrelevant. This indicates that prompt control alone cannot fully solve failure cases caused by ambiguous evidence, distractor passages, or model instability.

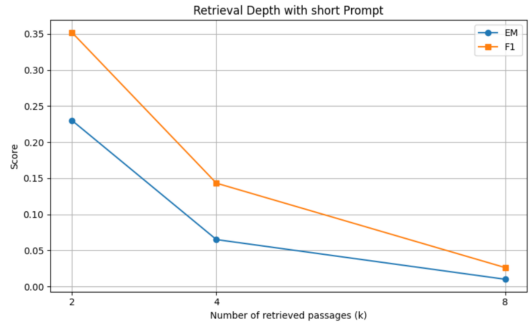

Figure 2: Comparison of EM and F1 across different retrieval depths. This figure should illustrate whether additional retrieved context helps or hurts answer quality when using the best prompt variant.

6.3 Tradeoffs

These experiments highlight a tradeoff between informativeness and evaluation alignment. Baseline prompts encourage more natural answers, but these often hurt exact-match evaluation. Stricter prompts improve answer compactness but may reduce fluency or cause brittle behavior if the model over-compresses the answer.

Similarly, increasing retrieval depth provides more evidence but can also overwhelm the model with noisy or redundant context.

7 Reflection

If I had more time, I would explore three next directions.

First, I would add answer post-processing or extraction to trim generated text down to the most likely span. This would likely improve Exact Match more directly than prompt changes alone.

Second, I would test reranking or hybrid retrieval. Some failures appear to stem from near-miss retrieval cases where the relevant evidence is present but not ranked prominently enough.

Third, I would investigate whether a smaller extractive QA model or answer-span selection stage could be placed after generation. This would create a hybrid system where generation helps reasoning but extraction helps formatting.

The biggest lesson from this part of the project is that generation control matters. RAG is not only about retrieving the right passages; it is also about ensuring the model uses those passages in a way that aligns with the task and metric.

8 Conclusion

This final project investigated a simple but meaningful improvement to a Wikipedia-based RAG pipeline: modifying prompt design to control answer form. By keeping retrieval fixed and varying only the prompt instruction, I showed that concise prompting improves F1 relative to a baseline RAG prompt on a 200-example development subset. Exact Match remained low because the model frequently produced paraphrases or explanatory text rather than exact spans, but the qualitative analysis suggests that prompt engineering still changes the system in a useful direction.

Overall, the results reinforce a key takeaway from the full project: both retrieval quality and prompt design matter in RAG systems, and even small generation-side changes can measurably affect downstream performance.

Limitations

This project focuses on a relatively small 200-example development subset, so results should be interpreted as exploratory rather than definitive. In addition, the final prompt experiments were evaluated on a generative model whose outputs are highly sensitive to formatting and decoding behavior. This makes Exact Match especially harsh and may understate improvements that are visible in F1 and in qualitative examples.

Another limitation is that the retrieval component was not fundamentally changed in the main experiment. As a result, improvements attributable to better evidence selection or reranking are not captured here. Finally, because the model sometimes ignores instruction constraints, prompt engineering alone cannot fully guarantee exact answer-span behavior.

Ethics Statement

This project uses large language models and retrieved web-scale text for question answering experiments. Such systems may produce hallucinated, misleading, or biased outputs even when prompted carefully. Although this work is purely evaluative and limited to a course setting, any deployment of similar systems would require more robust validation, stronger safety controls, and clearer uncertainty handling than what is implemented here.

Acknowledgments

I acknowledge the use of course materials, Pyserini, Hugging Face Transformers, Jupyter, and AI coding/writing assistance tools used during development and debugging. The structure and stylistic inspiration for the paper organization and appendix presentation were informed by the attached ACL-style research paper example.

References

A Prompts

The final project instructions require all prompts to be included in the appendix. The main prompts used in the final experiments are listed below.

A.1 Baseline Prompt

Context: [retrieved passages]

Question: [question text]

Answer the question using the provided context.

Answer:

A.2 Short-Answer Prompt

Context: [retrieved passages]

Question: [question text]

Answer the question using the provided context. Respond with ONLY the final answer phrase. Do not include explanations or extra words.

Answer:

A.3 Exact-Span Prompt

Context: [retrieved passages]

Question: [question text]

Answer the question using the provided context. Copy ONLY the exact answer span from the context. Do not write a full sentence. Do not include explanation.

Answer:

A.4 Best-Guess Prompt

Context: [retrieved passages]

Question: [question text]

Answer the question using the provided context. Give your best short answer even if the context is incomplete. Do not say you cannot find the answer. Respond with only the final answer phrase.

Answer:

B Implementation Details

- Retrieval library: Pyserini / Lucene BM25
- Corpus: local Wikipedia index
- Development subset size: 200 examples
- Main retrieval depth: $k = 4$
- Additional retrieval-depth analysis: $k \in \{2, 4, 8\}$
- Evaluation metrics: EM and F1

C Qualitative Error Examples

This appendix section should include 3–5 representative examples showing:

- a case where the baseline prompt was too verbose
- a case where the short-answer prompt improved token overlap
- a case where retrieval failed even with a better prompt

A suggested format is:

Question: ...
Gold Answer: ...
Baseline Prediction: ...
Improved Prediction: ...
Commentary: ...